CAHIER DE CHARGE

❖ **Acteurs**

- Administrateur
- Directeur des Etudes
- Secrétaire
- **ENTITES PRINCIPALES**
- Gestion des Utilisateurs
- Gestion des Étudiants
- Gestion des Formations
- Gestion des UE
- Gestion des ECUE
- Gestion des Inscriptions
- Gestion des Notes

FONCTIONNALITES ET CAS D'UTILSATION

➢ **Fonctionnalités de l'application**

1. Authentification et gestion des accès
2. Gestion des utilisateurs
3. Gestion des formations
4. Gestion des UE et ECUE
5. Gestion des étudiants
6. Gestion des inscriptions
7. Gestion des notes
8. Gestion des résultats académiques
9. Gestion des documents académiques
10. Traçabilité et historique des actions

➢ **Cas d'utilisation**

Fonctionnalité : Gestion des formations

- Créer une formation

- Modifier une formation
- Consulter une formation
- Supprimer une formation
-

Fonctionnalité : Gestion des UE et ECUE

- Créer une UE
- Modifier une UE
- Consulter une UE
- Supprimer une UE
- Créer une ECUE
- Modifier une ECUE
- Consulter une ECUE
- Supprimer une ECUE
- Associer une ECUE à une UE

Fonctionnalité : Gestion des étudiants

- Ajouter un étudiant
- Supprimer un étudiant
- Modifier un étudiant
- Consulter un étudiant
- Rechercher un étudiant

Fonctionnalité : Gestion des notes

- Saisir les notes
- Modifier les notes
- Consulter les notes
- Supprimer les notes

Fonctionnalité : Gestion des inscriptions

- Créer une inscription

- Modifier une inscription
- Consulter une inscription
- Supprimer une inscrition

Fonctionnalité : Gestion des résultats académaiques

- Générer le relevé de notes d'un étudiant par semestre
- Générer les notes par ecue
- Générer le relevé de notes anuel d'un étudiant

REGLES DE METIER

- Une formation est constituée de plusieurs UEs.
- Une UE est composée de plusieurs ECUEs.
- Un étudiant n'est inscrit qu'à une seule formation.
- Un étudiant s'inscris à une formation.
- Une inscription concerne une seule année académique.
- Pour une ECUE on a 2 notes de contrôle continu, une note d'examen terminal.
- La moyenne d'une ECUE est calculé avec 40% pour la moyenne des contrôle continu ou une note de control continu et 60% pour la note d'examen terminal.
- Si un étudiant n'a que la note d'examen terminal il est considéré comme sa moyenne.
- La note d'une UE est la moyenne des notes de ces ECUEs en considérant leur crédit.
- Une UE est validée si sa note est supérieure ou égale à 10.
- Si un étudiant n'a pas validé une UE il peut aller au rattrapage des ECUEs.

- Une UE peut être validé aussi si la moyenne des notes de rattrapage de ces ECUEs est supérieure ou égale à 10.
- Un étudiant a validé son année académique s'il a validé au moins 70% des UEs.

SPÉCIFICATIONS TECHNIQUES

1. Architecture générale

SchoolSoft est conçu selon une architecture client-serveur séparant l'interface utilisateur de la logique métier.

L'architecture est composée de trois couches principales :

- **Frontend** : application web développée avec React.js et Vite ;
- **Backend** : API REST développée avec Laravel ;
- **Base de données** : MySQL/MariaDB.

Les échanges entre le frontend et le backend sont effectués au format JSON via le protocole HTTP/HTTPS.

2. Technologies utilisées

Composant	Technologie
Frontend	React.js
Outil de développement frontend	Vite
Backend	Laravel 12
Langage backend	PHP 8.2 ou supérieur
API	REST / JSON
Base de données	MySQL / MariaDB
ORM	Eloquent
Authentification	Laravel Sanctum
Gestion des rôles	Spatie Laravel Permission
Tests	PHPUnit
Gestion de versions	Git / GitHub

3. Frontend

L'interface utilisateur est développée avec **React.js**.

React permet de construire l'application sous forme de composants réutilisables.

L'organisation du frontend peut notamment suivre la structure suivante :

```
src/
├── components/
├── pages/
├── services/
├── hooks/
├── layouts/
├── routes/
├── utils/
└── App.jsx
```

Services

Les communications avec l'API sont centralisées dans des services dédiés.

Exemples :

```
authService.js
formationService.js
ueService.js
ecueService.js
noteService.js
etudiantService.js
```

Cette organisation permet de séparer les appels API de l'affichage des composants.

4. Backend

Le backend est développé avec **Laravel 12** et expose une API REST.

L'organisation suit principalement l'architecture MVC de Laravel :

```
app/
├── Http/
│   ├── Controllers/
│   ├── Requests/
│   └── Resources/
├── Models/
├── Services/
└── ...
```

Controllers

Les contrôleurs reçoivent les requêtes HTTP et coordonnent les opérations nécessaires.

Exemples :

```
AuthController
FormationController
UEController
ECUEController
UE-ECUEController
NoteController
StudentController
InscriptionController
```

Form Requests

Les `FormRequest` sont utilisés pour centraliser la validation des données reçues par l'API.

Exemples :

```
RegisterRequest
LoginRequest
FormationRequest
NoteRequest
InscriptionRequest
```

Services

Les traitements métier complexes doivent être placés dans des classes de service afin d'éviter de surcharger les contrôleurs.

Exemples :

```
CalculNoteService
ResultService
```

5. Base de données

La base de données relationnelle utilisée est **MySQL/MariaDB**.

Les principales entités du système comprennent notamment :

- Utilisateur ;
- Étudiant ;
- Formation ;
- UE ;
- ECUE ;
- Note ;
- Inscription ;

Les relations entre les entités sont gérées à l'aide des clés primaires et étrangères.

Les migrations Laravel sont utilisées pour créer et modifier la structure de la base de données.

6. Authentification et autorisation

L'authentification de l'API est assurée par **Laravel Sanctum**.

Les utilisateurs doivent s'authentifier avant d'accéder aux ressources protégées.

Les autorisations sont gérées avec **Spatie Laravel Permission**.

Les accès sont définis selon les rôles et permissions de l'utilisateur.

Exemples de rôles :

```
Administrateur
DE
Secrétaire
```

Les permissions permettent de contrôler précisément les opérations accessibles à chaque rôle.

7. API REST

Les fonctionnalités principales sont exposées à travers des endpoints REST.

Exemples :

```
POST   /api/auth/register
POST   /api/auth/login
POST   /api/auth/logout
GET    /api/auth/profile

GET    /api/formations
POST   /api/formations/create
GET    /api/formations/{id}
PUT    /api/formations/edit/{id}
DELETE /api/formations/delete/{id}

GET    /api/ues
POST   /api/ues/create

GET    /api/ecues
POST   /api/ecues/ create

GET    /api/notes
POST   /api/notes/ create
```

```
PUT    /api/notes/{id}

GET    /api/results
GET    /api/etudiants/{id}/results
```

Les réponses de l'API sont retournées au format JSON.

8. Gestion des notes

Le système doit respecter les règles suivantes :

- Une note est comprise entre **0 et 20**.
- Les notes sont associées à un étudiant et à un ECUE.
- Les notes peuvent être de différents types, notamment devoir et examen.
- Les coefficients définissent la pondération des différentes évaluations.
- La moyenne est calculée automatiquement.
- Une UE est validée lorsque la moyenne obtenue est supérieure ou égale à **10/20**.
- La saisie et la modification des notes sont limitées aux utilisateurs autorisés.
- La validation définitive des résultats est effectuée par le responsable habilité.
- Un étudiant valide son année académique lorsqu'il a validé au moins 70 % des UE prévues pour l'année académique.
- Le système doit permettre d'enregistrer les notes de rattrapage
- des ECUE concernés.
- Lorsqu'une UE n'est pas validée à la session normale, les ECUE concernés peuvent être présentés au rattrapage conformément aux règles pédagogiques définies.
- Le système doit calculer la moyenne de rattrapage et determiner automatiquement si l'UE est validée.

9. Gestion des résultats

Le système doit permettre de :

- calculer automatiquement la moyenne de chaque ECUE ;

- calculer la moyenne de chaque UE en fonction des crédits ;

- déterminer les UE validées ;

- gérer les résultats de rattrapage ;

- déterminer la validation annuelle ;

- générer les relevés semestriels ;

- générer le relevé annuel.

10. Sécurité

Les mesures de sécurité comprennent :

- hachage des mots de passe ;
- authentification des utilisateurs ;
- contrôle des rôles et permissions ;
- validation des données côté serveur ;
- protection des routes sensibles ;
- utilisation de HTTPS en production ;
- protection des informations sensibles dans les variables d'environnement ;
- journalisation des opérations importantes ;
- interdiction d'exposer les mots de passe et tokens dans les logs.

Les informations sensibles doivent être stockées dans le fichier `.env` et ne doivent pas être versionnées dans Git.

11. Tests

Le projet doit comporter des tests automatisés avec PHPUnit.

Deux catégories principales sont utilisées :

Tests unitaires

Ils permettent de tester indépendamment les services et règles métier.

Exemples :

```
CalculNoteServiceTest
ResultServiceTest
PaymentServiceTest
```

Tests fonctionnels

Ils permettent de tester le comportement de l'API.

Exemples :

```
AuthTest
FormationTest
UETest
ECUETest
NoteTest
PaymentTest
```

Les tests doivent notamment vérifier :

- l'authentification ;
- les permissions ;
- la validation des données ;
- les opérations CRUD ;
- le calcul des notes ;
- la validation des UE ;
- les opérations financières.

14. Diagrammes techniques

Les principaux diagrammes du système sont fournis séparément dans le dossier :

```
docs/
└── diagrammes/
    ├── class_diagram.png
    └── use_case_diagram.png
```

Le **diagramme de classes** représente les principales entités du système ainsi que leurs relations.

Le **diagramme de cas d'utilisation** représente les acteurs du système et les fonctionnalités auxquelles ils peuvent accéder.

15. Évolutivité

L'architecture doit permettre l'ajout progressif de nouvelles fonctionnalités sans modifier profondément les modules existants.

Les évolutions possibles comprennent notamment :

- gestion multi-site ;
- notifications par email ou SMS ;
- consultation des résultats par les parents ;
- génération avancée de relevés et attestations ;
- gestion des paiemets
- tableaux de bord statistiques ;
- application mobile.